Master the Claude compact cheatsheet

STEP 1 Foundations

1.1 Meet Claude: Models, Tokens, Projects and Skills

What Claude is, in plain words Claude is an AI model made by Anthropic. You write to it in normal language and it writes back. It can read and write text, look at images and PDFs, write and run code, and use tools you connect to it.

The Claude models and when to pick each Claude comes in a family of models. Bigger models think more deeply. Smaller ones are faster and cheaper. All four current models read up to 1 million tokens at once and can write up to 128 thousand tokens in one answer.

Tokens and the context window Claude does not read letters or whole words. It reads tokens , which are small pieces of text. A short word is usually one token. A long word may be two or three.

How a conversation really works Claude has no hidden memory between messages. Each time you press send, the app sends the whole conversation so far , and Claude reads all of it again before replying.

Your first useful prompt A good prompt says who you are , what you have , what you want and what shape the answer should take . Here is one a home baker could send on any Sunday.

claude.ai Projects: instructions plus knowledge A Project in claude.ai is a folder for chats about one topic. It has two places to put information:

Skills: teach Claude a task once A Skill is a folder with a file called SKILL.md . The file has a short name and description at the top and the steps for one task below. Claude reads only the name and description at first. When a request matches, it opens the full file.

The map of this subject Each tool and file you will learn has one job. Hover or tap a box to see where it lives.

Safe habits from day one

TRAP Model names and prices change over time. Before you build something that costs money, check the models page in the Claude docs.

1.2 Prompt Engineering That Works

Be clear and direct Anthropic's own advice is to treat Claude like a brilliant but new employee . It is very capable, but it does not know your customers, your style or what "good" means to you.

Say why, not only what Claude follows a rule better when it knows the reason. A bare rule covers one case. A reason lets Claude handle cases you did not think of.

Show examples Examples are one of the most reliable ways to control format and tone. This is called few shot or multishot prompting.

Use XML tags to separate the pieces A prompt often mixes instructions, background, examples and data. Put each kind in its own tag, such as <instructions> , <context> or <notes> . There are no magic tag names. Pick clear ones and use them the same way every time.

Give Claude a role One sentence about who Claude is for this task changes tone and focus. In the API this goes in the system prompt . In claude.ai it goes in project instructions. In Claude Code the closest place is CLAUDE.md or an output style.

Control the format of the answer Tell Claude the exact shape: a numbered list, a table, JSON, a word limit, a closing line. Showing the shape works even better.

Let Claude think before answering For sums, comparisons and decisions, ask Claude to work through the problem first. Tags make it easy to keep the working separate from the final answer, so a script can read just the answer.

Prompt templates for daily work When you send the same kind of prompt every day, turn it into a template with blanks in double braces. Fill the blanks each time. Later, the same template becomes a skill (chapter 7) or a line in a script (chapter 11).

Chain prompts: draft, review, refine Big jobs go better in steps. The most useful chain is self correction : one prompt drafts, one checks the draft against a list, one rewrites. You can read each step and stop if something is off.

A recipe card for any prompt

TRAP If all your examples look alike, Claude may copy accidental details too, such as always starting with the same word. Vary them.

1.3 Context Engineering: The Right Information at the Right Time

Context engineering in one sentence Prompt engineering is about how you ask . Context engineering is about everything Claude can see when it answers: instructions, files, tools, past messages and notes.

Why more is not always better Claude's attention is a limited budget . Every extra token takes a little of it. As the window fills, the model gets slightly worse at recalling any one fact. Anthropic calls this context rot .

Write instructions at the right altitude A system prompt, CLAUDE.md or SKILL.md can be too low (a long list of brittle if this then that rules) or too high (vague advice that assumes Claude knows your business).

Tools and examples are context too Every tool Claude has comes with a name and description that sit in the context. Good tool sets are small , do not overlap and have clear names . If a human could not say which of two tools fits a job, Claude cannot either.

Just in time retrieval Do not paste everything up front. Give Claude light pointers such as file paths, a list of document titles or a search tool, and let it load details only when a question needs them.

Long tasks: compaction, notes and subagents Some jobs run for hours and outgrow any window. Three techniques keep them on track.

Context tools in Claude Code Claude Code gives you direct control over the window.

A context checklist for any task

STEP 2 Claude Code and its files

2.1 Claude Code: First Steps

What Claude Code is Claude Code is Claude working as an agent inside a folder on your computer. It can read your files, search them, edit them, run commands such as tests, and use git. You describe the goal in plain words.

Install and start, step by step

How it works: the agent loop For each request, Claude repeats a simple loop: gather context (search and read), take action (edit, run a command), check the result , then decide the next step. It stops when the task is done or it needs you.

Permission modes: how much Claude may do alone The permission mode decides which actions Claude can take without asking. Press Shift+Tab to switch during a session, or set defaultMode in settings.json (chapter 8).

Explore, plan, code, commit Anthropic's recommended workflow for anything bigger than a one line fix has four steps. The first two happen in plan mode , so nothing changes until you agree.

Stop, undo and come back later

The commands you will use most Two more shortcuts: type @ and a file name to point Claude at a file, and press the up arrow for your earlier requests.

The .claude folder: where everything lives Claude Code reads settings and instructions from two places: your project (shared with your team through git) and your home folder (just for you, in every project). The rest of this module fills in each file.

TRAP When Claude asks, read the command. "Yes, and don't ask again" saves a rule to `.claude/settings.local.json`, so think before choosing it for commands like `git push`.

TRAP Checkpoints cover edits Claude made with its file tools. They do not undo things outside your folder, such as a database change or a message sent by a command. Use git as well.

2.2 CLAUDE.md and Memory: What Goes Where

What CLAUDE.md is for CLAUDE.md is a plain Markdown file of instructions Claude Code reads at the start of every session . Write down what you would otherwise explain again and again: how to run the project, the house rules, the traps.

Where CLAUDE.md files live Claude Code loads these from the folder you start in and every folder above it , joined from the top down. A CLAUDE.md inside a subfolder loads later, when Claude reads or edits a file in that subfolder.

What to put in, and what to leave out This table comes from Anthropic's best practices guide. For every line, ask: would removing this cause Claude to make mistakes? If not, cut it.

Three real CLAUDE.md files Each file is short, specific and written for its own audience. Copy the one closest to your work and change the details.

Your personal files: ~/.claude/CLAUDE.md and CLAUDE.local.md Some instructions are about you , not the project. Keep them out of the team file.

Split a long file with @ imports Write @path/to/file anywhere in a CLAUDE.md to pull another file in. Paths are relative to the file that contains the import. Imported files may import others, up to four hops deep. A path inside backticks is not imported.

AGENTS.md and other tools Some teams already have an AGENTS.md file for other coding tools. Claude Code reads it as project instructions only when there is no CLAUDE.md , .claude/CLAUDE.md or CLAUDE.local.md in your folder or above it. If you have both, only the CLAUDE.md files load.

Auto memory: notes Claude writes for itself Besides your CLAUDE.md , Claude Code keeps its own notes per project. When you correct it ("use PowerShell, not bash"), it may save that so you do not have to repeat it next week.

Create, check and improve it

TRAP Claude treats `CLAUDE.md` as GUIDANCE, NOT ENFORCEMENT . It usually follows it, but it can slip. For "must never happen" rules, use a hook (chapter 8).

TRAP Imports help you organise, but they do not save context: imported files load at launch too. To load something only for certain files, use a path rule (chapter 6). To load it only for a task, use a skill (chapter 7).

2.3 Rules: The .claude/rules Folder and Path Rules

First, the name: there is no rules.md Many guides and videos talk about a "rules.md" file. Claude Code does not read a file with that name. What it does have is a folder of rule files:

Create your first rules, step by step Claude Code finds every .md file in .claude/rules/ , including subfolders like backend/ . A rule without paths loads at launch with the same priority as .claude/CLAUDE.md .

Path rules: load only for matching files Add YAML front matter with a paths list. The rule then loads only when Claude reads, writes or edits a file that matches one of the patterns. paths is the only field rules use. Any other field is ignored.

Ready to copy rule files Rules are not only for code. Any team that keeps files in a folder can use them.

Set up QA and developer rules, with screenshots This is the real setup of the rule files from the next two parts, in a small Python shop app. Both pictures are real terminal output.

Rule files for a QA engineer Testers repeat the same team standards in every chat: our ID format, no time.sleep , check the status code first. Put each standard in a rule with paths , and it appears exactly when Claude works on that kind of file.

Rule files for a developer Developers need two kinds of rules: a few that apply to all code, and some that matter only in risky places such as database migrations.

Personal rules in ~/.claude/rules/ Rules in your home folder apply to every project on your machine. They load before project rules. Neither overrides the other, so if they disagree Claude may follow either. Keep them consistent.

Share one set of rules across many projects The rules folder follows symlinks, so a company can keep its standards in one place and link them into every repository.

Rules, CLAUDE.md, skills or hooks?

When a rule does not seem to work

TRAP A path rule joins the conversation when its file is touched, so `/compact` summarises it away with everything else. It loads again the next time Claude touches a matching file. If a rule must survive compaction, remove its `paths` or move it to the root `CLAUDE.md`.

2.4 Skills: Build SKILL.md Step by Step

What a skill is A skill is a folder with a file called SKILL.md . The top of the file has a short description of when to use it. The rest has the steps . Claude Code, claude.ai and the API all use this format.

Build your first skill in three steps The line !`git diff HEAD` is dynamic context : Claude Code runs the command first and pastes the output into the skill, so Claude reads your real changes.

Set up a skill in Claude Code, with screenshots Here is the whole setup for a real QA skill, pick-regression , one screen at a time. The terminal pictures are real output from a real run. Use the numbers or the Back and Next buttons to move between steps.

Where skills live and how they get their name

The front matter: what goes at the top Every field is optional, but description matters most: Claude reads it to decide when to use the skill. Put the main use first and include the words people actually type.

Arguments and live data Text typed after the skill name becomes its arguments. Use $ARGUMENTS for all of it, $0 , $1 for each word (counting from zero), or names listed in arguments . Lines with !`command` are replaced by the command's output before Claude sees the skill.

Supporting files and scripts Keep SKILL.md short (under 500 lines) and put detail in other files in the same folder. Link to them from SKILL.md so Claude knows when to open them. Scripts are run , not read, so a long script costs almost nothing in context.

Who can start a skill Use disable-model-invocation for anything with side effects, like deploying or sending a message. You do not want Claude deciding to deploy because the code "looks ready".

Skills for daily work The best first skills are jobs you already do every week. Here are two more to copy.

Skills for a QA engineer's day A tester repeats the same kinds of work every day: reading stories, writing test cases, logging bugs and deciding what to retest. Each of those is a perfect skill, because the steps are the same every time and only the input changes.

Skills for a developer's day A developer's day has its own loop: pick up an issue, fix it, review the change, ship it. Three skills cover that loop.

Skills in claude.ai and the API The same folder can be uploaded to claude.ai as a ZIP, or sent through the Skills API. Uploads follow the open Agent Skills format, which is stricter than Claude Code:

Make skills that work every time

TRAP `allowed-tools` skips permission prompts while the skill runs. Read the `allowed-tools` of skills in a repository before you trust it.

2.5 settings.json, Permissions and Hooks

The settings files and which one wins Settings are plain JSON files. Each one has a different reach. When two files set the same key, the higher one in this list wins. Lists such as permission rules are combined from all files.

Permission rules: allow, ask and deny A rule names a tool, and optionally what it may touch: Bash(pytest *) , Read(./.env) , Edit(app/**) , WebFetch(domain:example.com) . A * matches anything.

Ready to copy settings files A safety detail: defaultMode values auto and bypassPermissions are ignored in project files, so a repository cannot switch your safety checks off. Allow rules in a project file apply only after you trust the folder. Deny and ask rules apply straight away.

Hooks: code that runs on events A hook is a command Claude Code runs at a fixed moment, such as just before a tool runs or when Claude is waiting for you. Unlike CLAUDE.md , a hook always runs.

Hook 1: block dangerous commands Claude sees the "Blocked" message and tries a safer way, or asks you.

Hook 2: protect secret files, with a JSON answer Instead of an exit code, a PreToolUse hook can print JSON with permissionDecision set to deny , ask or allow , plus a reason Claude will read.

Hook 3: add live facts to prompts A UserPromptSubmit hook can add text to Claude's context with additionalContext , nested inside hookSpecificOutput . Here it tells Claude about a release freeze, but only when the prompt is about deploying.

Hook 4: after every edit, log, format or notify Two more from the official guide, ready to paste:

Check and debug

TRAP Settings files are strict JSON. A comment or a trailing comma breaks the file. Add the `$schema` line to get autocomplete and checks in your editor, and run `/status` to confirm what loaded.

TRAP Look at the `/bin/rm` line. Pattern rules match the command as written, so `Bash(rm *)` does not catch `/bin/rm` or `bash -c 'rm ...'`. Use deny rules for clear cases and a HOOK for anything that truly must be stopped.

TRAP Hooks run with your user's permissions. Read every hook in a repository you clone before you trust the folder.

2.6 Subagents and Output Styles

What a subagent is A subagent is a helper that Claude can hand a task to. It has its own context window , its own instructions and its own allowed tools. It does the work, then sends back only a short result . Your main conversation stays small and focused.

Built-in subagents you already have Claude uses these on its own. You will see a row in the transcript such as Explore(Find the payment code) .

Create your first subagent, step by step

Every field in an agent file Multi word fields use camelCase, such as maxTurns . A misspelled field is ignored without an error.

Ready to copy agents

Calling a subagent

Agents that remember With memory set, the agent keeps its own notes folder and gets better over time, for example learning which support issues keep coming back.

Output styles: change how Claude talks for a whole session An output style changes Claude's role, tone and answer format for the session. Four built-in styles sit on top of the Default:

2.7 MCP: Connect and Build Servers

What MCP is MCP (Model Context Protocol) is an open standard that lets Claude use tools and data from other programs: your issue tracker, a database, a calendar, or a script you wrote yourself. One program, the MCP server , offers the tools.

Connect a server, step by step

Set up MCP in Claude Code, with screenshots Here is the expenses server from this chapter added to Claude Code for real, then a shared server added for the whole team. The terminal pictures are real output, with the home folder shown as ~ .

Scopes and the .mcp.json file When the same name is in several scopes, local wins, then project, then user. Claude Code asks once before it uses servers from a project's .mcp.json .

Build your own server in Python Here is a server that saves expenses. Claude will be able to say "Saved team lunch for ₹850" because it called your code.

What travels between Claude and a server Under the hood, MCP is a conversation of small JSON messages (JSON-RPC). For a stdio server they go one per line on stdin and stdout. First a handshake, then Claude asks for the tool list, then it calls a tool.

Using MCP inside Claude Code Every MCP tool gets a full name mcp__<server>__<tool> . Use that name in permission rules, in a skill's allowed-tools , in an agent's tools and in hook matchers.

The same servers in Claude Desktop The Claude Desktop chat app reads its own file, claude_desktop_config.json . Open it from Settings, then Developer, then Edit Config. On macOS it is in ~/Library/Application Support/Claude/ and on Windows in %APPDATA%\Claude\ .

MCP in claude.ai: connectors, with screenshots In claude.ai an MCP server is called a connector . Popular services are in the Connectors Directory. Your own server can be added as a custom connector .

TRAP Custom connectors are not checked by Anthropic. Only add servers from people and companies you trust, because a connector can read and sometimes change data with your permissions.

2.8 Daily Automation: Plugins, Scripts and GitHub

Plugins: share your whole setup in one folder By now you have skills, agents, hooks and MCP servers. A plugin puts them in one folder so a whole team can install them with one command. Use plain .claude/ files for one project, and a plugin when several projects or people need the same setup.

claude -p: use Claude in scripts The -p flag runs Claude Code once, prints the answer and exits. That makes it a normal command you can put in shell scripts, Makefiles and scheduled jobs.

Get data back, not just text Ask for JSON in a fixed shape with --json-schema . The data arrives in the structured_output field, ready for the next step of your script.

Claude in GitHub Actions

Run it on a schedule With a prompt input, the action runs on any event, including a timer. GitHub schedules use cron lines in UTC.

Daily automation recipes

STEP 3 Building with the API

3.1 The Claude API Step by Step

Get a key and install the SDK The Claude API lets your own program talk to Claude. You pay per token used, separate from a Claude.ai subscription.

Your first call, line by line

System prompts and conversations The system prompt sets the role and rules for the whole chat. The API does not remember anything between calls, so your code keeps the history and sends it every time.

Choose a model and know the cost All four have a 1 million token context window and up to 128K output tokens. Haiku 5.5 costs $0.50 / $2.50 for prompts over 100K tokens.

Streaming: show words as they arrive Long answers take time. Streaming sends the text piece by piece, so your app can show it at once. Use it for anything long, because it also avoids request timeouts.

Get JSON your code can trust With output_config.format and a JSON schema, the reply is valid JSON in exactly that shape. No more "please answer only in JSON" and hoping.

Effort and prompt caching The stand-in API does not cache, so it reports 0 for both. Against the real API the first call writes the policy to the cache and calls in the next 5 minutes read it.

When things go wrong

Choose a model and know the cost

cost = (input tokens × input price + output tokens × output price) ÷ 1,000,000

3.2 Build Your Own Agents

What an agent really is An agent is Claude plus tools plus a loop . Claude reads the goal, asks for a tool, your code runs it and sends back the result, and this repeats until Claude says it is done.

Define tools Claude can use A tool is a name, a description and an input schema . Claude chooses tools by reading the descriptions, so write them for a new colleague: what it does, when to use it and what each input means.

Build the loop by hand, step by step Here is a shopkeeper's assistant. It checks the rice stock, sees it is low and places an order, in two rounds of tool use.

Less code with the tool runner The SDK's tool runner writes the loop for you. Decorate a normal Python function with @beta_tool : the type hints become the schema and the docstring becomes the description.

The Claude Agent SDK: Claude Code as a library The Agent SDK gives your program the same tools, loop, permissions, hooks, skills and MCP support as Claude Code. You do not write tools for reading files or running commands. You just say which ones are allowed.

Which way to build your agent

STEP 4 Capstone

4.1 Capstone: Which File Holds What

Every file, and what goes in it

The real names behind the names people use

Decide where an instruction belongs Ask four questions in order. Must it happen every time? Use a hook. Is it about what Claude may do? Use permissions. Is it a procedure or only for some files? Use a skill or a path rule. Otherwise it is a short line in CLAUDE.md .

A complete project, reviewed Here is a whole shop project with every kind of Claude file. Read the tree, then the three main files, and look for what a careful reviewer would flag.

Build a homework helper, step by step This is small enough for a child and a parent to build together in 15 minutes.

Your next level, one week at a time

STEP 5 Certification prep

5.1 Claude 101 and Certification Prep

What Claude 101 is and how you earn the certificate Claude 101 is a free online course from Anthropic Academy , hosted on Skilljar. Its own description says it teaches you to "use Claude for everyday work tasks, understand core features, and explore resources for more advanced learning". No coding is needed.

Module 1: Meet Claude and get better results What Claude is. Claude is a large language model. It has read a huge amount of text and writes answers by predicting what text should come next. Two facts follow:

Module 1: Chat, Cowork and Code on your desktop The Claude Desktop app has three ways to work. They share one home screen and message box.

Module 2: Projects, artifacts and skills

Memory, past chats and incognito Claude can remember things across chats, and you stay in control of it.

Module 3: Connectors, enterprise search and Research Connectors let Claude read from and act in the apps you already use, such as Google Drive, Gmail, Slack or your own tools. They are built on MCP, the same standard Chapter 10 teaches.

Module 4: Claude in action, role by role The course shows how different teams use the same features. These examples follow the same idea.

Module 4: Other ways to work with Claude The last module, What's next? , points to more Anthropic Academy courses. Chapters 4 to 13 of this subject cover the same ground step by step.

The Claude Certified Associate exam (CCAO-F) If you want a credential with a pass mark, Anthropic has a proctored exam called Claude Certified Associate, Foundations , code CCAO-F .

What each exam domain tests, with one tip each Chapter 1 has the full model table. Chapters 2 and 3 cover prompting and context in depth.

Exam style scenarios, worked through The exam asks what you would do in a work situation. Here is how to think through three typical ones.

A seven day study plan

Cheat sheet: numbers and menu paths

> **Cheat sheet: numbers and menu paths**
> up to 10 ×

TRAP On Team and Enterprise plans, an owner decides whether memory is available. Memory is not available for organisations with HIPAA, public sector or custom data retention agreements.

TRAP Third party guides say registration is open only to people at organisations in the Claude Partner Network, using a work email from that organisation. Rules can change, so read the certifications page on the Anthropic Partner Academy before you plan your date.